Анализ примеров задач и стратегия решения без LLM

Выводы по загруженным примерам задач (search, data_analysis, math, code) и корректировка плана под бюджет 100к токенов.

Что видно из примеров

Задачи небольшие и явно шаблонные: в каждой инструкции написано, какими инструментами решать и что именно записать в `answer.txt`. В примерах четыре типа:

Тип	Пример	Как решается
search	«В каком городе штаб-квартира работодателя X?»	Цепочка файлов: `person_x.md` → поле `Employer` → `company_y.md` → `Headquarters`
data_analysis	«Сколько units по product 'widget' за месяцы 7–8?»	Один фильтр и сумма в pandas
math	Цепочка операций «разделить, оставить долю, удвоить»	Простой цикл (получается 389, как в `expected`)
code	`mylib.py` + `test_mylib.py`	Две ошибки: `reverse_words` сортирует вместо разворота, `total` вычитает вместо сложения

Поля задач: `reward` от 1 до 5, `check` равен `exact_str` или `exact_int`. Поле `expected` в примерах есть, в финальном наборе его, скорее всего, не будет.

По поиску: для Sergei Garcia и Irina Smith карточек людей среди загруженных файлов нет, поэтому эти два ответа проверить нельзя. Остальное сходится: Alice Smith → Granite AI → Osaka, та же цепочка из двух переходов, что и в задаче про Sergei Garcia.

Главный вывод для плана

Почти все эти задачи можно решать **без LLM, за 0 токенов**: шаблонный парсер вытаскивает параметры из текста, а локальный код считает ответ.

- **math:** регулярные выражения разбирают шаги («split into N», «doubled») и считают их по порядку.
- **data_analysis:** парсится продукт, диапазон месяцев и колонка, затем `pandas` суммирует.
- **search:** парсится имя и целевое поле, затем идёт цепочка поиска по `kb/` через grep и разбор строк `Key: value`.
- **code:** здесь нужна LLM, но дёшево: один вызов `noreason` с `mylib.py` и текстом упавших тестов, на выходе исправленный файл.

Поэтому бюджет 100к нужен почти только для кодовых задач и для задач с нераспознанным шаблоном, куда уходит запасной вызов LLM. При равенстве баллов выигрывает тот, кто потратил меньше токенов, так что решатель без LLM даёт ещё и преимущество на тай-брейке.

Риск: формулировки в скрытом наборе могут отличаться от примеров. Поэтому схема такая: сначала шаблонный решатель, а если шаблон не распознан или результат пустой, то LLM.

Невыполнимые задачи

В поиске это ловится естественно: если имени нет ни в одном файле `kb/` или цепочка обрывается (нет карточки компании), задача невыполнима, и токены на неё тратить не надо. Для данных это отсутствие нужной колонки или файла, для кода — отсутствие тестов или модуля.

Что дальше

1. Создать каталог `solvers/` с решателями math, data и search и локальный прогон на этих примерах.
2. Добавить LLM-обёртку только для кодовых задач и запасного пути.
3. Сгенерировать собственный мини-`kb/` в тех же форматах для проверки поиска (недостающие `.md` присылать не обязательно).
4. Обновить основной план, добавив раздел про шаблонные решатели.